Graphic Era deemed to be University DEHRADUN

# PROJECT AND TEAM INFORMATION

## USB Malware Activity Monitoring System Using DSA Concepts and Object Oriented Programming Concepts

| |
|---|
| USB Malware Activity Monitoring System |

| | |
|---|---|
| *TeamName:* | |
| **Team member 1 (Team Lead)** | *Divyanshu Panwar*<br>*2028175*<br>*2510370085@geu.ac.in*<br>  |
| **Team member 2** | *Abhay Rai*<br>*2028610*<br>*2510370531@geu.ac.in*<br>  |
| | |
| | |

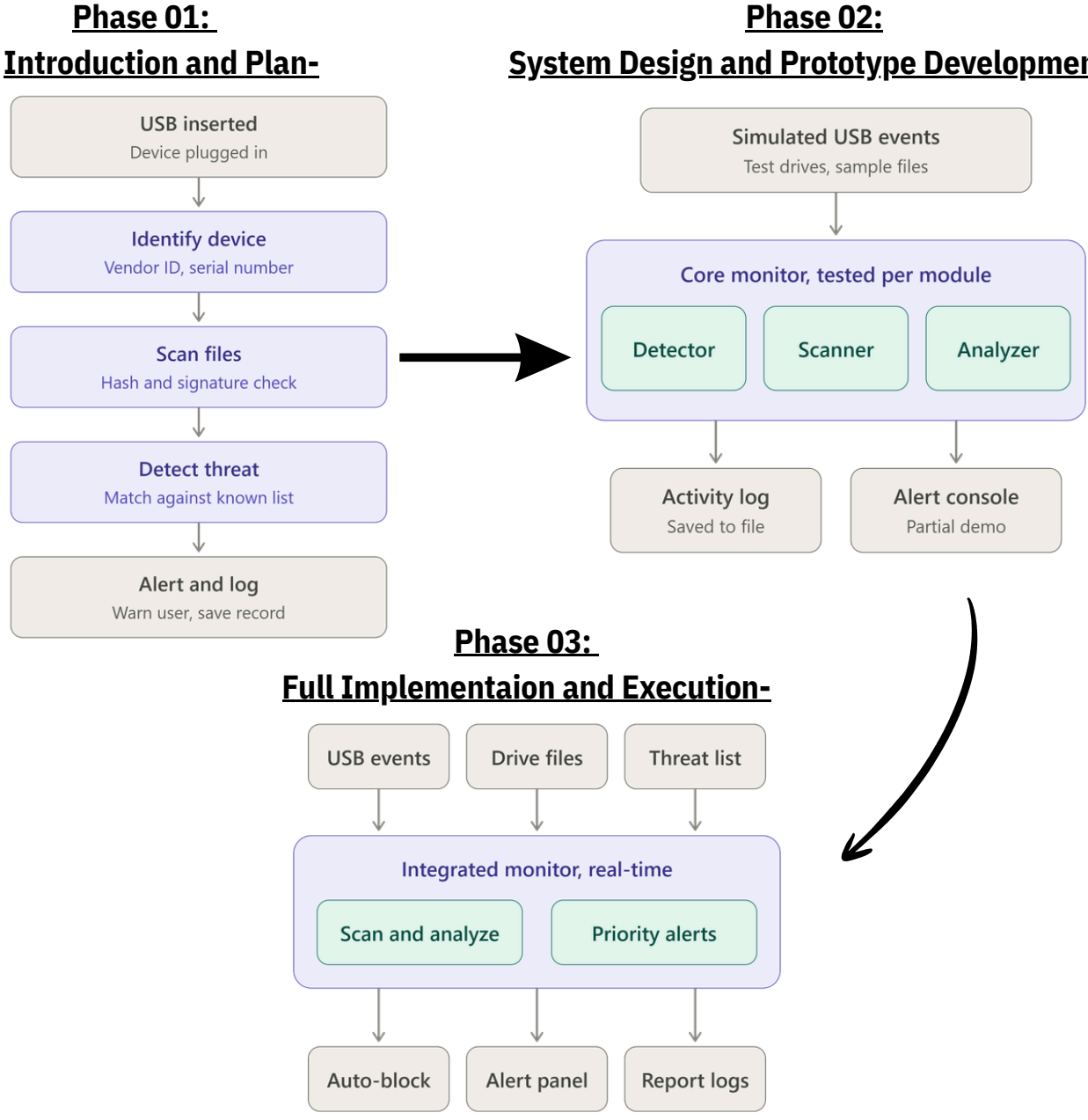
# PHASE-II PROJECT PROGRESS DESCRIPTION

## Project Abstract

The USB Malware Actibity Monitor is a security based project designed to monitor and identify suspicious activities performed using a USB device connected to a computer. USB is a commonly used device to transfer data , but it can also introduce suspicious files into the system. The project aims to provide a basic monitoring system that observes USB related activities and helps user to identify malcious behaviour.

The Project Combines DSA Topics and Object-Oriented-Programming Concepts. DSA will be used for system level operations like data handling, file operations and monitoring functionality. OOPs concepts like classes, objects, encapsulation and inheritance will be used to organize different components of system and make project easier to maintain.

## Updated Project Approach and Architecture



## Tasks Completed (7 pts)

| Task Completed | TeamMember |
|---|---|
| Problem Identified, Phase (01,02) Architecture Diagram, Concepts Identified, Working Understanding, Presentation for Phase 02 and Report for Phase 02 | 1.Divyanshu Panwar-2028715<br>2.Abhay Rai-2028610 |

## Challenges/Roadblocks (7 pts)

1.Building or finding a good threat list.
2.Incomplete knowledge about topics, threats and algorithms used in the project.
3.Choosing the right and better data structure for each module used in project.
4.Time and scope limits for a student project.
5.Scanning large drives without freezing the program.
6.Needing admin rights to read device details or block devices.
7.Detecting USB events reliably across Windows and Linux.
8.Hash matching only catches known threats

## Tasks Pending (7 pts)

| Task Pending | Team Member (to complete the task) |
|---|---|
| Testing the project, Integrating different modules into 1 single project, fixing bugs (if they occur), speed up the scanning process, report and presentation for Phase 03 and complete the project with a final demo. | 1.Divyanshu Panwar-2028715<br>2.Abhay Rai-2028610 |

# Project Outcome/Deliverables

1.The project delivers a working monitor that protects a computer from malicious USB devices. When a drive is plugged in, the system identifies the device, scans its files, and compares them against a known threat list. Anything suspicious is ranked by severity, shown as an alert, and saved in a log. Serious threats can be blocked automatically.

2.The system is built from three tested modules: Detector, Scanner and Analyzer. Each was first validated with simulated USB events and safe sample files, then integrated into a real-time monitor with an alert panel and report logs.

3.The project shows how C++ OOP and DSA concepts solve a real security problem. Hash tables give fast threat lookup, queues manage incoming events and scan jobs, and a priority queue makes sure high-risk alerts are handled first.

# Progress Overview

- Problem and objectives defined.
- Architecture diagrams made for all three phases.
- Technologies and data structures chosen (C++ OOP, hash table, queue, priority queue).
- Code for different modules is made.

**Pending**

- Join all modules into one system.
- Test the full flow and the edge cases.
- Prepare the final demo, report and presentation.
- Write the test report, with the problem list.
- Test each module separately and fix the bugs.
- Make the alert console and activity log.
- Set up fake USB events and safe test files, such as the EICAR file.

## Codebase Information

For the detailed information for the project you may consider the link below:
https://github.com/divyanshupanwar7895-lang/PBL.git

The **USB Malware Activity Monitor** is a security-oriented system designed to monitor activities performed by USB devices when they are connected to a computer.

The project combines C programming for system-level and file-related operations with C++ Object-Oriented Programming (OOP) for modular and structured system development.

## Testing and Validation Status (2 pts)

| TestType | Status (Pass/Fail) | Notes |
|---|---|---|
| Tests To Done | Tests To Done | Tests To Done |

## Deliverables Progress (2 pts)

**1.Architecture diagram:** a picture of the Phase 2 system.
**2.Presentation slides:** slides for the Phase 2 review.
**3.Design document:** a plan showing how the three parts (Detector, Scanner, Analyzer) are built and connected.
**4.Data flow diagram:** a picture showing how information moves between the modules.
**5.Class diagram:** a chart of how the C++ classes are organised.
**6.Design document:** a plan showing how the three parts (Detector, Scanner, Analyzer) are built and connected.
**7.Progress report:** a short summary of what is done and what comes next.